A Semantic Reduction from Conditional Safety to Local Proofs

FRÉDÉRIC DABROWSKI, Univ.Orléans, INSA Centre Val de Loire, LIFO EA 4022, France

We study conditional safety of reactive programs over infinite input streams and unbounded semantic domains. The target property says that every input stream satisfying an assumption must produce only executions satisfying a safety guarantee. Such properties are global by nature, whereas deductive verification is local and transition-based. We identify a canonical local proof interface for this mismatch. Starting from the explicit product of the program with assumption and guarantee safety automata, we localize global violations to dangerous product steps, extract semantic clauses from these steps, and package these clauses into relational Hoare triples over ticks. We prove that this reduction is sound: validity of all generated triples, together with semantic truth of the user invariants on admissible executions, implies conditional safety. We also prove two relative-completeness results for the reduction itself: global correctness implies admissible-run validity of all generated safety triples, and, under true user invariants on admissible executions, admissible-run validity of all generated triples. The development is formalized in Rocq. Concrete validation mechanisms are treated only as later instantiations of this abstract proof interface.

CCS Concepts: • **Software and its engineering** → **Formal methods**; *Software verification*; • **Theory of computation** → **Logic and verification**.

Additional Key Words and Phrases: reactive programs, conditional safety, local proofs, relational Hoare logic, safety automata

ACM Reference Format:
Frédéric Dabrowski. 2026. A Semantic Reduction from Conditional Safety to Local Proofs. *Proc. ACM Program. Lang.* 1, , Article 1 (January 2026), 27 pages.

1 INTRODUCTION

Safety requirements for reactive programs are naturally stated over infinite traces. In the conditional setting considered here, one does not ask that every environment behavior be safe, but rather that every *admissible* input stream lead to a safe execution. This yields properties of the form

$$\text{if the input satisfies } A, \text{ then the execution satisfies } G,$$

where A and G denote safety properties. Such formulations are standard in assume/guarantee reasoning and in synchronous verification [4, 19, 23].

The technical difficulty is that the property is global, whereas deductive verification proceeds locally. The state spaces of the programs of interest are not necessarily finite, since they manipulate semantic domains such as integers, records, or memories with user-defined structure. Direct finite-state reasoning is therefore unavailable at the program level. At the same time, safety properties themselves admit canonical finite representations through safety automata and their standard constructions [1, 17, 21]. The problem is to connect these two levels without collapsing the semantics of the program into a backend-specific verification-condition encoding.

This question is not merely an engineering concern. If the mathematical model is phrased too directly in terms of a concrete verification backend, then the semantic content of the reduction becomes hard to isolate: soundness depends on tool-specific encodings, and completeness statements become difficult to interpret. Conversely, if the model stays entirely at the level of global trace properties, then it fails to explain why local proof obligations should suffice at all. A satisfactory

Author's address: Frédéric Dabrowski, Univ.Orléans, INSA Centre Val de Loire, LIFO EA 4022, Orléans, France, frederic.dabrowski@univ-orleans.fr.

account should therefore expose an intermediate layer that is simultaneously semantic, local, and backend-independent.

The non-trivial point is therefore not the mere existence of safety automata, nor the observation that local verification conditions are useful in practice. The non-trivial point is to identify the semantic object that makes the local proof layer both inevitable and backend-independent. The thesis of this paper is that the explicit product between the program and the two safety automata is exactly this object. Once the product has been fixed, dangerous steps, semantic clauses, and relational triples are no longer ad hoc proof artifacts: they become the canonical local interface of the global conditional-safety semantics.

The contribution is a small semantic theory organized around three central results. First, validity of all generated relational triples, together with semantic truth of the user invariants on admissible executions, implies conditional safety. Second, global correctness implies admissible-run validity of the generated NoBad triples. Third, under the additional assumption that user invariants are semantically true on admissible executions, all generated triples are admissibly valid. The first theorem justifies the reduction as a sound proof method; the second and third explain why the reduction generates semantically legitimate local proof objects rather than arbitrary artifacts.

The core theory is independent of any particular tool. In practice, one may start from a specification language such as a safety fragment of LTL, compile it to safety automata, and discharge the generated triples with an external backend such as Why3 [13]. These are instantiations of the abstract triple layer, not part of the central mathematical result.

This perspective matters. The point is not only that a concrete verifier can be organized around products and local triples. The point is that conditional safety admits a semantic reduction whose local proof objects are determined by the product semantics itself. This is the level at which the paper claims generality.

The paper should therefore be read primarily as a theorem about a reduction, not as a description of a verification pipeline. The pipeline perspective reappears only once the abstract triple layer has been fixed and justified.

Scope of the result. The paper proves a reduction theorem, not a complete verification pipeline. The core result stops at the validity of relational triples generated from the product semantics, together with the semantic truth of user-supplied invariants on admissible executions. Concrete choices such as formula syntax, encoding into a verification-condition language, proof search, or solver orchestration belong to later instantiation layers. This separation is deliberate: it keeps the semantic theorem stable while allowing multiple backends to refine it.

Contributions. The paper makes four contributions. It identifies the explicit product as the canonical semantic object that localizes conditional-safety reasoning for reactive programs over unbounded domains. It defines semantic clauses and relational Hoare triples as the local proof interface induced by this product. It proves soundness and relative-completeness theorems for the resulting reduction, making explicit that soundness relies only on generated-triple validity and on the semantic truth of user invariants on admissible executions. Finally, it provides a mechanized Rocq development whose public structure mirrors the mathematical argument.

The point is not merely that a verifier can be organized around products and triples. The point is that, once the product has been fixed, the local proof layer is mathematically determined by the semantics rather than chosen as a backend convenience.

Structure of the paper. Section 2 recalls the standard synchronous and automata viewpoints used by the paper. Section 3 gives a running example. Section 4 defines the reactive model, conditional safety, and the explicit product. Section 5 introduces semantic clauses and relational triples. Section 6

states and proves the main results. Section 7 explains how a concrete tool can instantiate the abstract theory. Section 8 discusses related work and Section 10 concludes.

2 BACKGROUND

The paper combines four standard viewpoints that are often studied separately. First, we adopt the synchronous view of reactive computation [4, 19]: execution proceeds in discrete ticks, and each tick transforms a current internal configuration, a memory, and an input into a next internal configuration, a next memory, and an output. This one-tick organization is crucial for the rest of the paper, because it suggests from the outset that the relevant local proof objects should relate current and next tick contexts rather than arbitrary long prefixes.

This synchronous reading also makes clear why a purely finite-state model-checking account is not sufficient for the class of programs considered here. The program state may contain integers, records, or user-defined memory structures ranging over unbounded domains. For such systems, direct finite exploration is unavailable in general without additional abstraction, symbolic restrictions, or specialized infinite-state techniques [6, 9]. This is the setting in which the paper places its reduction: the specification side retains a finite representation through safety automata, while the program side remains symbolic and can range over unbounded domains.

Second, the temporal properties of interest are safety properties over infinite traces [1]. Safety is exactly the fragment for which global violation admits a finite witness: once a bad event has occurred, no extension of the execution can repair it. This is why deterministic safety automata are the natural finite representation of the temporal component of the specification. They compress the relevant prefix information into a finite state and expose failure through a distinguished bad state. In the conditional setting studied here, one automaton reads inputs and captures admissibility, while the other reads input/output observations and captures the guarantee. In concrete instances, these automata can come from a safety fragment of temporal logic through standard safety-automaton constructions [17, 21]. For safety properties, this deterministic presentation is not an ad hoc implementation choice: it is the representation that makes finite bad prefixes explicit, and therefore matches the intended reduction to local proofs.

Third, the intended proof style is deductive and local. The goal is not to reason directly on whole traces, but to extract local proof obligations whose validity suffices for the global safety statement. For this reason, the target proof objects are not command-level Hoare triples over an imperative syntax, but relational triples over semantic ticks. Their role is analogous to the role of verification conditions in ordinary Hoare logic [10, 20]: they state what must hold before and after one local step. The difference is that the "command" is here the semantic relation induced by one reactive tick.

The reduction developed in the rest of the paper should therefore be read as a bridge between these four ingredients: synchronous ticks provide the right granularity for local reasoning, safety automata provide a finite handle on the temporal part of the specification, conditional safety explains why two automata are needed, and relational triples provide the deductive interface through which the global property can be established locally.

The intended concrete instances belong to the same broad family as synchronous intermediate languages inspired by Lustre and observer-based compilation [7, 19, 24]. In such languages, a program is already organized around an explicit memory, an explicit control structure, and tick-wise transitions. This explains why the abstract model of the paper gives such a central role to internal modes and local transitions.

This ordering of ideas is also the reading strategy of the paper itself. We first fix the synchronous and safety-theoretic setting, then introduce the product that localizes global failures, and only afterwards turn this product into local proof objects.

3 OVERVIEW

This section gives the informal picture that the formal development will make precise. The reader should keep four objects in mind: the program, the two safety automata, the explicit product, and the local triple extracted from a dangerous product step.

Consider a reactive program with an integer control state $s \in \{Init, Run\}$, an integer memory cell m, an integer input x, an integer input r used as a Boolean reset flag, and an output y. The initial control state is *Init*, and *Run* is absorbing. The intended discipline is that $r \in \{0, 1\}$ and that reset ticks carry the distinguished reinitialization value $x = 0$. At each tick, if $r = 1$ then the program emits 0, stores 0 in memory, and moves to *Run*; otherwise it emits either 0 while $s = Init$ or the current memory while $s = Run$, stores the current input for the next tick, and again moves to *Run*. In pseudo-code:

$$\begin{array}{l}\textbf{if } r = 1 \textbf{ then } y := 0;\ m' := 0;\ s' := Run\\ \textbf{else if } s = Init \textbf{ then } y := 0;\ m' := x;\ s' := Run\\ \textbf{else } y := m;\ m' := x;\ s' := Run\end{array}$$

The running specification is:

$$\begin{array}{l}A \equiv G((r = 0 \vee r = 1) \wedge (r = 1 \Rightarrow x = 0)),\\ G_1 \equiv G(r = 1 \Rightarrow y = 0),\\ G_2 \equiv X\,G((r = 0 \wedge \mathsf{prev}(r) = 1) \Rightarrow y = 0),\\ G_3 \equiv X\,G((r = 0 \wedge \mathsf{prev}(r) = 0) \Rightarrow y = \mathsf{prev}(x)).\end{array}$$

The aggregate guarantee is the conjunction of G_1, G_2, and G_3. The user invariant attached to 'Run' is likewise split into two cases:

$$(\mathsf{prev}(r) = 1 \Rightarrow m = 0) \wedge (\mathsf{prev}(r) = 0 \Rightarrow m = \mathsf{prev}(x)).$$

This same structure will later be instantiated in a concrete specification language.

For readability, the figures below abbreviate the main labels of the running example as follows:

$$\varphi_A \triangleq (r \in \{0, 1\}) \wedge (r = 1 \Rightarrow x = 0),$$

$$\psi_{rst} \triangleq (r = 1 \Rightarrow y = 0), \qquad \psi_{first} \triangleq (r = 0 \wedge \mathsf{pre}(r) = 1 \Rightarrow y = 0),$$

$$\psi_{delay} \triangleq (r = 0 \wedge \mathsf{pre}(r) = 0 \Rightarrow y = \mathsf{pre}(x)).$$

The proof method does not start by generating verification conditions directly from the source program. Instead, it forms the product of the program with a safety automaton for the assumption and a safety automaton for the guarantee. A concrete tick then determines a concrete program transition and advances both automata. The key local notion is that of a *dangerous step*: a product step that remains admissible for the assumption automaton but moves the guarantee automaton into its bad state.

This dangerous step is the pivot of the reduction. It is the local event that witnesses a global guarantee violation. Once such a step has been identified, the reduction does two things. First, it extracts a semantic safety clause NoBad stating that the current tick does not realize that step. Second, it packages this clause into a local relational triple whose precondition contains exactly the coherence information needed to make the safety clause meaningful.

The coherence layer is part of the reduction, not an external proof heuristic. Some coherence triples transport user invariants. Others transport the fact that the current concrete tick is coherent with the current abstract product state. Safety triples and coherence triples therefore live in the same local proof world: the latter provide the local context in which the former can exclude dangerous concrete steps.

The point of the example is not that its local proof obligations are difficult, but that the reduction separates three levels cleanly. At the trace level, the property is conditional safety. At the product

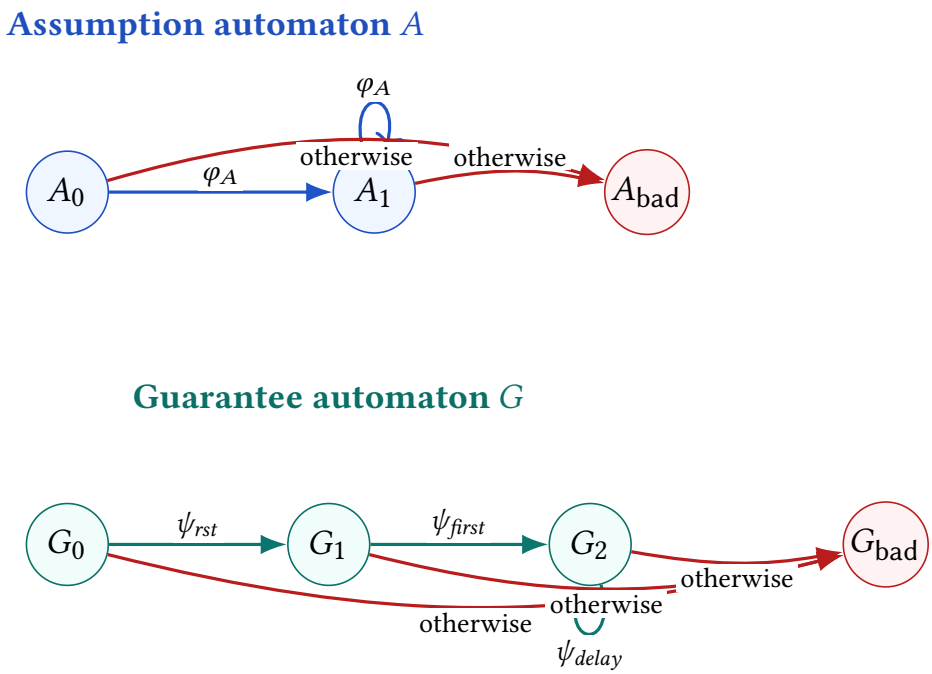

Fig. 1. Safety automata for the running example. The assumption automaton tracks the reset discipline, while the guarantee automaton distinguishes three residual modes: reset ticks (G_0), the first non-reset tick after reset (G_1), and the ordinary one-step delay case (G_2). In both automata, the red "otherwise" transitions collect the locally unsafe cases and lead to the bad state.

level, violations become dangerous local steps. At the triple level, dangerous steps become local proof objects whose preconditions contain exactly the coherence information needed for local reasoning. The same pattern will be used in the general theory, and Section 7 will return to this example in a concrete specification language.

A concrete dangerous case. Suppose the current tick is a non-reset tick with $\mathsf{prev}(r) = 0$, so the active guarantee branch is G_2 and the expected output is $\mathsf{prev}(x)$. If the local context records a transition whose concrete output disagrees with that value, then the guarantee automaton moves to its bad state while the assumption automaton may still remain admissible, since the Boolean/reset discipline can still hold. This yields a dangerous product step. The associated safety clause simply says that the current context must not realize this step. The corresponding safety triple is local, but its justification still depends on two kinds of coherence facts: the user invariant that explains the meaning of m in the current reset mode, and the automaton coherence fact that identifies the current abstract product state.

Figure 1 explains which abstract modes of the assumption and guarantee are relevant on the running example. The labels φ_A, ψ_{rst}, ψ_{first}, and ψ_{delay} are exactly the local predicates that distinguish the reset case, the first non-reset tick after reset, and the ordinary delay case. Figure 2 then isolates the single dangerous product edge used throughout the proof sketch. Here q_A and q_G denote the current states of the assumption and guarantee automata in the source product state. The facts $\mathsf{Coh}^{\mathsf{cur}}_{p_{\mathrm{src}}}$ and $\mathsf{Inv}_{\mathsf{Run}}$ justify the precondition of the local safety triple; $\mathsf{Bad}(p)$ classifies the edge

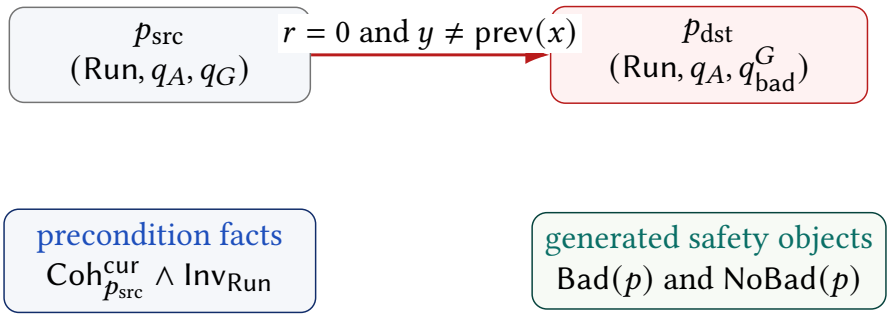

Fig. 2. Relevant fragment of the explicit product for the running example. The source state combines the current control state with the current assumption and guarantee automaton states. The red edge is dangerous because its assumption component remains admissible while its guarantee component reaches the bad state. If a concrete tick matches this edge, the generated triple $h^{\text{NoBad}}(p)$ is activated.

as dangerous, whereas $\text{NoBad}(p)$ is the semantic clause generated from this classification and embedded into the triple $h^{\text{NoBad}}(p)$.

What the overview already shows. Even before the formal definitions, the example exhibits the full proof architecture. A global violation of the guarantee is localized to one concrete tick. The local context of that tick identifies a dangerous product step. This step generates a semantic safety clause. The clause is embedded into a local triple whose precondition is built from coherence information. The rest of the paper turns this informal picture into a general semantic reduction theorem:

$$\text{valid generated local triples} \Longrightarrow \text{conditional safety.}$$

Reduction at a glance

1. A global guarantee violation yields a dangerous product step.
2. That step yields one local safety clause $\text{NoBad}(p)$.
3. The clause is packaged into one local safety triple $h^{\text{NoBad}}(p)$.
4. Coherence triples justify the local preconditions of $h^{\text{NoBad}}(p)$.
5. Valid generated triples, together with true user invariants on admissible executions, imply conditional safety.

The formal development below simply turns each line of this summary into a definition or theorem. The running example keeps reappearing so that the reader never has to reconstruct the intended local picture from the abstract definitions alone. The only semantic fact not reconstructed from generated triples themselves is the truth of user invariants on admissible executions; this dependency will be isolated explicitly in the meta-theory.

4 SEMANTIC MODEL

The purpose of this section is not to introduce yet another operational model, but to isolate the smallest semantic structure needed by the reduction. We need a notion of reactive tick, a notion of safety automaton over infinite streams, and a product that makes one local step carry exactly the information needed by the local proof interface. Each definition below should therefore be read as introducing one piece of this interface.

4.1 Reactive programs

Intuition. The reduction does not depend on a concrete source syntax. What matters is only that one reactive tick can be observed semantically: from a current internal state, a current memory,

and a current input, the program chooses a transition and produces a successor state, a successor memory, and an output. The formal notion of control state is introduced just below; before that point, it can be read simply as the abstract mode in which the program currently operates. We package this directly as semantic data.

Definition 1 (Reactive program)**.** A reactive program is a tuple

$$P = (S, M, I, O, T, s_0, \mathsf{select}, \mathsf{enabled}, \mathsf{dst}, \mathsf{upd}, \mathsf{out})$$

where S, M, I, O, and T are respectively the control-state, memory, input, output, and transition spaces; $s_0 \in S$ is the initial control state; and

$$\begin{aligned} \mathsf{select} &: S \times M \times I \to T, \\ \mathsf{enabled} &: T \times S \times M \times I \to \mathsf{Prop}, \\ \mathsf{dst} &: T \times S \to S, \\ \mathsf{upd} &: T \times M \times I \to M, \\ \mathsf{out} &: T \times M \times I \to O. \end{aligned}$$

The selected transition is always enabled:

$$\forall s \in S, \forall m \in M, \forall i \in I, \quad \mathsf{enabled}(\mathsf{select}(s, m, i), s, m, i).$$

The theory is intentionally extensional: it does not assume a command syntax, a control-flow graph, or a verification-condition generator, but only the semantic data needed to describe one reactive tick. By construction, programs in this model are total at the level of one tick: for every current control state, current memory, and current input, the model provides a selected transition, a successor control state, a successor memory, and an output. The transition guard is represented by enabled. Since $\mathsf{select}(s, m, i)$ is required to return an enabled transition, every concrete tick already carries the fact that the guard of the selected transition holds. Later triples therefore do not need to repeat this guard as an extra ad hoc premise: it is already built into the semantic notion of a realized step.

Definition 2 (Execution from an initial memory)**.** Fix a program P, an input stream $u : \mathbb{N} \to I$, and an initial memory $m_0 \in M$. The execution of P from m_0 on u is the tuple of streams (s_u, m_u, t_u, o_u) defined by:

$$s_u(0) = s_0, \qquad m_u(0) = m_0,$$

and, for every $k \in \mathbb{N}$,

$$\begin{aligned} t_u(k) &= \mathsf{select}(s_u(k), m_u(k), u(k)), \\ o_u(k) &= \mathsf{out}(t_u(k), m_u(k), u(k)), \\ s_u(k+1) &= \mathsf{dst}(t_u(k), s_u(k)), \\ m_u(k+1) &= \mathsf{upd}(t_u(k), m_u(k), u(k)). \end{aligned}$$

Running example. For RESETTABLE-DELAY, the control state space is $S = \{\mathsf{Init}, \mathsf{Run}\}$, the memory space is the integer domain, the inputs are pairs (r, x) made of a reset flag and an integer, and the output is an integer y. On reset ticks ($r = 1$), the program emits 0 and stores 0. On non-reset ticks, it emits the current memory and stores the current input x, except for the first non-reset tick in state Init, where it emits 0 and moves to Run.

4.2 Conditional safety

Intuition. The program alone is not enough: the property is conditional. One automaton filters admissible inputs; the other checks whether the observable input/output behavior remains safe. Since we focus on safety, both automata are deterministic safety automata with a distinguished bad state.

Definition 3 (Deterministic safety automaton)**.** For an alphabet X, a deterministic safety automaton is a tuple

$$\mathcal{A} = (Q, q_0, q_{\text{bad}}, \delta)$$

where Q is a state space, $q_0 \in Q$ is the initial state, $q_{\text{bad}} \in Q$ is the bad state, and $\delta : Q \times X \to Q$ is a deterministic step function.

Thus safety automata are total at the level of one letter: for every current state and every current observation, the next automaton state is defined by the function δ.

Definition 4 (Run and safety acceptance)**.** Let $\mathcal{A} = (Q, q_0, q_{\text{bad}}, \delta)$ be a deterministic safety automaton over X, and let $w : \mathbb{N} \to X$ be an infinite word. The run of $\mathcal{A}$ on w is the stream $q_w : \mathbb{N} \to Q$ defined by

$$q_w(0) = q_0, \qquad q_w(k+1) = \delta(q_w(k), w(k)).$$

We say that $\mathcal{A}$ accepts w when

$$\forall k.\ q_w(k) \neq q_{\text{bad}}.$$

The assumption automaton is written

$$A = (Q_A, q_0^A, q_{\text{bad}}^A, \delta_A)$$

and reads inputs in I. The guarantee automaton is written

$$G = (Q_G, q_0^G, q_{\text{bad}}^G, \delta_G)$$

and reads observable input-output pairs in $I \times O$. For an internal run from m_0 on input stream u, the observable trace is

$$\tau_{m_0,u}(k) = (u(k), o_u(k)).$$

The conditional specification additionally assumes that the initial states of both automata are not already bad:

$$q_0^A \neq q_{\text{bad}}^A \qquad \text{and} \qquad q_0^G \neq q_{\text{bad}}^G.$$

Definition 5 (Conditional safety predicates)**.**

$$\mathsf{Avoid}_A(u) \triangleq \forall k \in \mathbb{N}.\ q_u^A(k) \neq q_{\text{bad}}^A,$$
$$\mathsf{Avoid}_G(u) \triangleq \forall m_0 \in M, \forall k \in \mathbb{N}.\ q_{m_0,u}^G(k) \neq q_{\text{bad}}^G.$$

Running example. In RESETTABLE-DELAY, the assumption automaton checks the input-side discipline $r \in \{0, 1\}$ and $r = 1 \Rightarrow x = 0$. The guarantee automaton checks three safety branches: reset ticks emit 0, the first non-reset tick after a reset emits 0, and ordinary non-reset ticks emit the previous non-reset input. Thus $\mathsf{Avoid}_A(u)$ means that the input stream respects the reset discipline, whereas $\mathsf{Avoid}_G(u)$ means that every execution on u respects all three branches of the intended delay behavior.

The distinction between assumption and guarantee is crucial. The assumption automaton constrains which inputs are semantically relevant, whereas the guarantee automaton constrains which

observable input/output traces are acceptable for the program. The core theorem is therefore a theorem of conditional safety, not of unconditional trace acceptance.

4.3 Explicit product

Intuition. At this point one could try to reason directly on executions and automaton runs, but that would keep the safety argument global. The product is introduced precisely to make one local tick carry the program state and the two automaton states at once. It is the object on which "dangerous local step" becomes a meaningful notion.

Definition 6 (Product state)**.** A product state is a triple

$$p = (s, q_A, q_G) \in S \times Q_A \times Q_G.$$

We write $\pi_S(p) = s$, $\pi_A(p) = q_A$, and $\pi_G(p) = q_G$ for its three projections.

Definition 7 (Product state along an execution)**.** Fix m_0 and u. Let q_u^A be the run of A on u, and let $q_{m_0,u}^G$ be the run of G on $\tau_{m_0,u}$. The product state at tick k is

$$\mathsf{pst}_{m_0,u}(k) \triangleq (s_u(k), q_u^A(k), q_{m_0,u}^G(k)).$$

Definition 8 (Product step)**.** A product step is a tuple

$$p = (p_{\mathrm{src}}, t, m, i, o, p_{\mathrm{dst}})$$

where p_{src} and p_{dst} are product states, $t \in T$, $m \in M$, $i \in I$, and $o \in O$.

Thus a product step records one program tick together with the current memory and observation, and with the source and target states induced in the product.

Definition 9 (Product-step target)**.** For a source product state $p_{\mathrm{src}} = (s, q_A, q_G)$, a transition $t \in T$, a memory $m \in M$, an input $i \in I$, and an output $o \in O$, we define

$$\mathsf{Target}(p_{\mathrm{src}}, t, m, i, o) \triangleq (\mathsf{dst}(t, s), \delta_A(q_A, i), \delta_G(q_G, (i, o))).$$

Definition 10 (Selected product step)**.** Fix $m_0 \in M$, an input stream u, and a tick $k \in \mathbb{N}$. The selected product step at tick k is

$$\mathsf{step}_{m_0,u}(k) \triangleq (\mathsf{pst}_{m_0,u}(k), t_u(k), m_u(k), u(k), o_u(k), \mathsf{pst}_{m_0,u}(k+1)).$$

Running example. In the running example, a product state records one control state (Init or Run), one assumption-automaton state, and one guarantee-automaton state. A product step for a non-reset tick in Run records not only the source and target automaton states, but also the current memory m, the current input (r, x) with $r = 0$, and the emitted output $y = m$. This is precisely the information needed to decide whether the guarantee branch "ordinary delay" is respected locally.

The explicit product is finite as soon as S, Q_A, and Q_G are finite. Memory and observations are not folded into the product state; they are carried only by local product steps and local tick contexts.

4.4 Local tick contexts and matching

Intuition. The product step is still an abstract object. To connect it to actual executions, we need a local semantic snapshot of one concrete tick. This is the role of the tick context: it records the current configuration, the current observation, the selected transition, and the immediate successor configuration. Matching is then the bridge between a concrete tick and an abstract product step.

Definition 11 (Tick context)**.** Fix m_0, u, and k. The tick context at tick k is

$$\mathsf{Ctx}_{m_0,u}(k) \triangleq (\mathsf{curS}, m_u(k), u(k), o_u(k), t_u(k), \mathsf{curA}, \mathsf{curG}, \mathsf{nextS}, m_u(k+1), \mathsf{nextA}, \mathsf{nextG})$$

with

$$\mathsf{curS} = s_u(k), \qquad \mathsf{curA} = q_u^A(k), \qquad \mathsf{curG} = q_{m_0,u}^G(k),$$
$$\mathsf{nextS} = s_u(k+1), \qquad \mathsf{nextA} = q_u^A(k+1), \qquad \mathsf{nextG} = q_{m_0,u}^G(k+1).$$

We reuse the symbols curS, curA, curG, nextS, nextA, nextG for the corresponding coordinate projections on arbitrary tick contexts.

This context is the local semantic object inspected by clauses and triples: it contains exactly the current configuration, the transition chosen at the current tick, and the immediate successor information needed to speak about the next product state. The context is deliberately local: the reduction never manipulates complete execution prefixes explicitly, but packages into one object the exact amount of concrete information that one local proof step is allowed to inspect.

Definition 12 (Matching)**.** Let

$$p = (p_{\mathrm{src}}, t, m, i, o, p_{\mathrm{dst}}).$$

We say that the tick context $\mathsf{Ctx}_{m_0,u}(k)$ matches p, written $\mathsf{Match}(\mathsf{Ctx}_{m_0,u}(k), p)$, when the following equalities hold:

$$p_{\mathrm{src}} = \mathsf{pst}_{m_0,u}(k), \qquad t = t_u(k), \qquad m = m_u(k),$$
$$i = u(k), \qquad o = o_u(k),$$

and

$$p_{\mathrm{dst}} = \mathsf{pst}_{m_0,u}(k+1).$$

The output component is essential: without it, the target guarantee state of the product step would not be determined by the local context. Matching also records the selected program transition itself. As a consequence, once $\mathsf{Match}(\mathsf{Ctx}_{m_0,u}(k), p)$ holds, the guard of the transition carried by p is already known to hold at the current tick, because the matched transition is exactly the one returned by select.

Definition 13 (Dangerous step)**.** A product step $p = (p_{\mathrm{src}}, t, m, i, o, p_{\mathrm{dst}})$ with $p_{\mathrm{dst}} = (s', q'_A, q'_G)$ is dangerous when

$$q'_G = q_{\mathrm{bad}}^G \qquad \text{and} \qquad q'_A \neq q_{\mathrm{bad}}^A.$$

We write $\mathsf{Bad}(p)$ for this predicate.

Definition 14 (Well-formed product step)**.** For

$$p = (p_{\mathrm{src}}, t, m, i, o, p_{\mathrm{dst}}),$$

we define $\mathsf{WFStep}(p)$ by

$$p_{\mathrm{dst}} = \mathsf{Target}(p_{\mathrm{src}}, t, m, i, o).$$

Thus $\mathsf{WFStep}(p)$ is a structural consistency condition on the step itself. Realization on a concrete execution is expressed separately by the matching predicate Match.

Running example. The canonical dangerous case is a non-reset tick in state Run where the assumption still accepts the current input, but the guarantee moves to its bad state because the emitted value does not agree with the previous non-reset input. In that case the product step is dangerous exactly because the assumption remains admissible while the guarantee fails.

Proposition 15 (Global violation yields a dangerous step)**.** *Let $m_0 \in M$, and let u be an input stream. If*

$$\mathsf{Avoid}_A(u) \qquad \textit{and} \qquad \exists j \in \mathbb{N}.\ q_{m_0,u}^G(j) = q_{\mathrm{bad}}^G,$$

then there exists a tick $k \in \mathbb{N}$ such that

$$\mathsf{Bad}(\mathsf{step}_{m_0,u}(k)).$$

This proposition is the semantic localization principle: it reduces a global violation of the guarantee to a concrete local event.

Proof idea. If the guarantee automaton reaches its bad state on an execution, then there is a first tick at which the bad state is entered. The current program state, the current automaton states, the selected program transition, and the current observation determine a concrete product step. By construction, its target guarantee state is bad. Since the assumption still holds on the run, the target assumption state is not bad. Therefore the realized product step is dangerous. □

5 REDUCTION TO LOCAL PROOFS

The reduction has two layers. First, dangerous product steps induce semantic clauses that describe what must be ruled out or propagated locally. Second, these clauses are assembled into relational triples, which are the actual local proof objects. The distinction matters: clauses carry the semantic content of the reduction, while triples provide the local proof interface.

5.1 Semantic clauses

Intuition. Clauses are the smallest semantic facts extracted from the product. Safety clauses prohibit dangerous concrete realizations. Invariant clauses record semantic information supplied by the specification. Coherence clauses say which abstract product state the current or next concrete tick corresponds to.

Definition 16 (Semantic clause)**.** A semantic clause is a predicate over tick contexts

$$C : \mathsf{Ctx} \to \mathsf{Prop}.$$

where Ctx denotes the ambient space of tick contexts.

We use three canonical families of clauses.

Definition 17 (Safety clause)**.** For a dangerous product step p, the safety clause generated by p is

$$\mathsf{NoBad}(p)(\mathsf{Ctx}) \triangleq \neg\mathsf{Match}(\mathsf{Ctx}, p).$$

Definition 18 (User-invariant clause)**.** For a control state $s \in S$, the user-invariant clause attached to s is a semantic predicate

$$\mathsf{Inv}_s : \mathsf{Ctx} \to \mathsf{Prop}.$$

In the abstract theory, these predicates are part of the specification rather than part of the program semantics.

Definition 19 (Automaton-coherence clauses)**.** Let $p = (s, q_A, q_G)$ be a product state. The current coherence clause and next coherence clause attached to p are the predicates

$$\mathsf{Coh}_p^{\mathsf{cur}}(\mathsf{Ctx}) \triangleq (\mathsf{curS}(\mathsf{Ctx}) = s \wedge \mathsf{curA}(\mathsf{Ctx}) = q_A \wedge \mathsf{curG}(\mathsf{Ctx}) = q_G),$$

$$\mathsf{Coh}_p^{\mathsf{next}}(\mathsf{Ctx}) \triangleq (\mathsf{nextS}(\mathsf{Ctx}) = s \wedge \mathsf{nextA}(\mathsf{Ctx}) = q_A \wedge \mathsf{nextG}(\mathsf{Ctx}) = q_G).$$

These clauses are semantic, not syntactic: they state directly that the current or next part of the tick context corresponds to a given abstract product state. This is why the automaton-coherence part of the theory does not require an extra correctness hypothesis about generated support formulas.

5.2 Relational Hoare triples

Intuition. The local proof interface is not a syntax-directed Hoare logic for commands. Instead, each generated proof obligation talks about one semantic tick. The precondition constrains the current context, the postcondition constrains the next one, and the target tells us which initial or one-step semantic situation the triple talks about.

The actual local proof objects are relational Hoare triples. A triple is attached either to the initial tick or to a concrete program transition.

Definition 20 (Relational Hoare triple)**.** A relational Hoare triple is a tuple

$$h = (\mathit{target}, \mathit{pre}, \mathit{post})$$

where *target* is either the distinguished initial tick or a program transition $t \in T$, and where *pre* and *post* are semantic clauses.

We use $\top$ and $\bot$ for the always-true and always-false semantic clauses.

Definition 21 (Transition relation of a program transition)**.** For a transition $t \in T$, we define

$$\begin{aligned}\mathsf{TransRel}_t(\mathsf{Ctx}, \mathsf{Ctx}') \triangleq\ & \exists m_0 \in M, \exists u : \mathbb{N} \to I, \exists k \in \mathbb{N},\\ & \mathsf{Ctx} = \mathsf{Ctx}_{m_0,u}(k) \wedge \mathsf{Ctx}' = \mathsf{Ctx}_{m_0,u}(k+1) \wedge t_u(k) = t.\end{aligned}$$

Definition 22 (Triple validity)**.**

$$\mathsf{Valid}((\mathit{init}, \mathsf{pre}, \mathsf{post})) \triangleq \forall m_0 \in M, \forall u : \mathbb{N} \to I,\ \mathsf{pre}(\mathsf{Ctx}_{m_0,u}(0)) \to \mathsf{post}(\mathsf{Ctx}_{m_0,u}(0)).$$

For a step triple with target transition $t \in T$, we define

$$\mathsf{Valid}((t, \mathsf{pre}, \mathsf{post})) \triangleq \forall \mathsf{Ctx}, \mathsf{Ctx}'.\ \mathsf{TransRel}_t(\mathsf{Ctx}, \mathsf{Ctx}') \wedge \mathsf{pre}(\mathsf{Ctx}) \Longrightarrow \mathsf{post}(\mathsf{Ctx}').$$

Definition 23 (Admissible-run triple validity)**.**

$$\begin{aligned}\mathsf{ValidAdm}((\mathit{init}, \mathsf{pre}, \mathsf{post})) \triangleq\ & \forall m_0 \in M, \forall u : \mathbb{N} \to I.\\ & \big(\mathsf{Avoid}_A(u) \wedge \mathsf{pre}(\mathsf{Ctx}_{m_0,u}(0))\big) \to \mathsf{post}(\mathsf{Ctx}_{m_0,u}(0)).\end{aligned}$$

For a step triple with target transition $t \in T$, we define

$$\begin{aligned}\mathsf{ValidAdm}((t, \mathsf{pre}, \mathsf{post})) \triangleq\ & \forall m_0 \in M, \forall u : \mathbb{N} \to I, \forall k \in \mathbb{N}.\\ & \big(\mathsf{Avoid}_A(u) \wedge t_u(k) = t \wedge \mathsf{pre}(\mathsf{Ctx}_{m_0,u}(k))\big)\\ & \Longrightarrow \mathsf{post}(\mathsf{Ctx}_{m_0,u}(k+1)).\end{aligned}$$

This is a relational form of Hoare reasoning rather than an imperative one. The triple does not interpret a command syntax. Instead, it constrains the semantic transition relation induced by one program tick. In set-theoretic terms, if R_t is the relation associated with transition t, then validity expresses that R_t maps the set denoted by the precondition into the set denoted by the postcondition. This distinction matters for readability as well as for theory: the paper is not proposing a new command language or a new verification-condition calculus, but identifying the semantic shape of the local proof obligations that any concrete backend will later refine. The guard of t is therefore exploited implicitly through $\mathsf{TransRel}_t(\mathsf{Ctx}, \mathsf{Ctx}')$: a triple for t is only required to hold on pairs of contexts linked by an actual semantic step labeled by t. In other words, the reduction reasons under the guard of t by restricting attention to realized steps, rather than by repeating that guard as a separate conjunct in every precondition.

Running example. For the dangerous non-reset Run-step of RESETTABLE-DELAY, the corresponding safety triple has a precondition saying: the current context matches this step, the current product

state is indeed the expected one, and the user invariant attached to Run holds. Its postcondition is $\bot$. This is the local proof object that excludes the concrete mismatch $y \neq \mathsf{prev}(x)$.

5.3 Generated triples

Intuition. The generated triples are not an arbitrary menu of local proof obligations. The initial triples establish the facts that must already hold at the first tick. The propagation triples say how these facts move from one tick to the next. The safety triples then use exactly these facts to rule out dangerous concrete realizations. The five families below should therefore be read as the minimal closure of the reduction under initialization, propagation, and local safety.

Definition 24 (Canonical generated triples)**.** Let

$$p_0 \triangleq (s_0, q_0^A, q_0^G)$$

be the initial product state. The reduction uses the following canonical triples:

$$h^{\mathsf{init}}_{\mathsf{Inv}} \triangleq (\mathit{init}, \top, \mathsf{Inv}_{\pi_S(p_0)}),$$
$$h^{\mathsf{init}}_{\mathsf{Coh}} \triangleq (\mathit{init}, \top, \mathsf{Coh}^{\mathsf{cur}}_{p_0}).$$

For every well-formed product step

$$p = (p_{\mathrm{src}}, t, m, i, o, p_{\mathrm{dst}}),$$

we define

$$h^{\mathsf{prop}}_{\mathsf{Inv}}(p) \triangleq \big(t,\ \lambda\mathsf{Ctx}.\ \mathsf{Match}(\mathsf{Ctx}, p) \wedge \mathsf{Inv}_{\pi_S(p_{\mathrm{src}})}(\mathsf{Ctx}) \wedge \mathsf{Coh}^{\mathsf{cur}}_{p_{\mathrm{src}}}(\mathsf{Ctx}),\ \mathsf{Inv}_{\pi_S(p_{\mathrm{dst}})}\big),$$
$$h^{\mathsf{prop}}_{\mathsf{Coh}}(p) \triangleq \big(t,\ \lambda\mathsf{Ctx}.\ \mathsf{Match}(\mathsf{Ctx}, p) \wedge \mathsf{Coh}^{\mathsf{cur}}_{p_{\mathrm{src}}}(\mathsf{Ctx}),\ \mathsf{Coh}^{\mathsf{cur}}_{p_{\mathrm{dst}}}\big).$$

For every dangerous well-formed product step p, we define

$$h^{\mathsf{NoBad}}(p) \triangleq \big(t,\ \lambda\mathsf{Ctx}.\ \mathsf{Match}(\mathsf{Ctx}, p) \wedge \mathsf{Inv}_{\pi_S(p_{\mathrm{src}})}(\mathsf{Ctx}) \wedge \mathsf{Coh}^{\mathsf{cur}}_{p_{\mathrm{src}}}(\mathsf{Ctx}),\ \bot\big).$$

The safety clause $\mathsf{NoBad}(p)$ is therefore not itself the postcondition of the safety triple. Rather, it is the semantic content enforced by the triple: if the precondition of $h^{\mathsf{NoBad}}(p)$ holds at a tick, then no successor context can exist, so the current context cannot realize the dangerous step p. The two propagation families transport the two local facts needed by the safety triples: user invariants and automaton coherence. These facts are therefore transported in the same local proof contexts rather than in two independent proof phases.

Definition 25 (Generated triples)**.**

$$\begin{aligned}\mathsf{Generated}(h) \triangleq\ & h = h^{\mathsf{init}}_{\mathsf{Inv}} \ \vee\ h = h^{\mathsf{init}}_{\mathsf{Coh}} \\ & \vee\ \exists p.\ \mathsf{WFStep}(p) \wedge h = h^{\mathsf{prop}}_{\mathsf{Inv}}(p) \\ & \vee\ \exists p.\ \mathsf{WFStep}(p) \wedge h = h^{\mathsf{prop}}_{\mathsf{Coh}}(p) \\ & \vee\ \exists p.\ \mathsf{WFStep}(p) \wedge \mathsf{Bad}(p) \wedge h = h^{\mathsf{NoBad}}(p),\end{aligned}$$

and

$$\mathsf{Generated}_{\mathsf{NoBad}}(h) \triangleq \exists p.\ \mathsf{WFStep}(p) \wedge \mathsf{Bad}(p) \wedge h = h^{\mathsf{NoBad}}(p).$$

Running example. For RESETTABLE-DELAY, the generated initialization triples establish the invariant and automaton coherence facts at the first tick. The propagation triples transport these facts across reset and non-reset transitions. Finally, the safety triples forbid exactly the dangerous local situations identified in the product, such as a non-reset Run step whose output disagrees with the delayed input expected by the guarantee automaton.

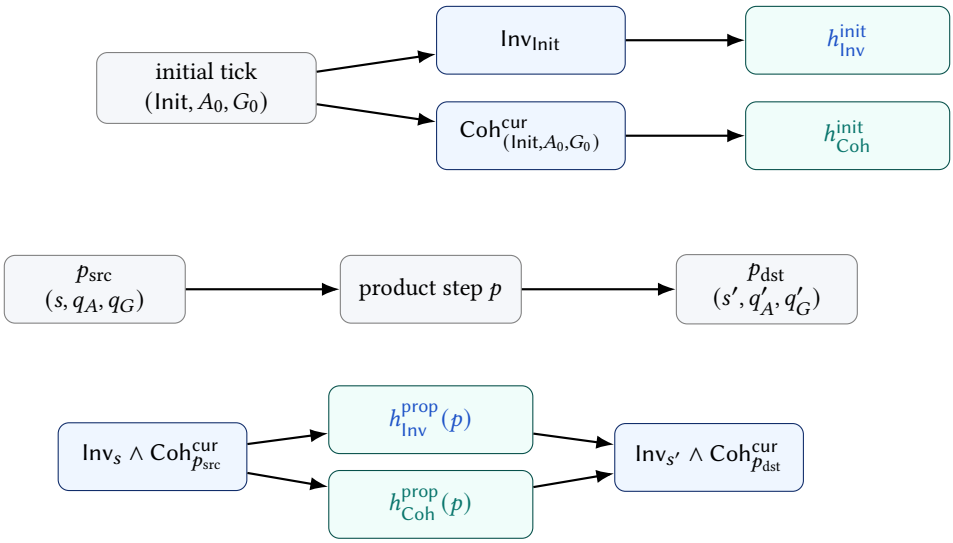

Fig. 3. Initialization and propagation triples.

It is useful to make the partition explicit. The reduction generates exactly five families:

$$
\begin{array}{rcl}
h^{\mathsf{init}}_{\mathsf{Inv}} & : & \text{initial user-invariant triple,} \\
h^{\mathsf{init}}_{\mathsf{Coh}} & : & \text{initial automaton-coherence triple,} \\
h^{\mathsf{prop}}_{\mathsf{Inv}}(p) & : & \text{propagation of the user invariant along } p, \\
h^{\mathsf{prop}}_{\mathsf{Coh}}(p) & : & \text{propagation of product-state coherence along } p, \\
h^{\mathsf{NoBad}}(p) & : & \text{local safety triple for a dangerous step } p.
\end{array}
$$

For the running example, the first four families establish and propagate the local facts needed to interpret the delayed-output requirement, while the last family excludes precisely the dangerous case highlighted in Figure 2.

This is the point at which the reduction becomes easy to read operationally. The two initialization triples explain how the local proof context starts; the two propagation families explain how that context survives from one tick to the next; and the safety family then uses exactly this accumulated local context to exclude the dangerous product steps singled out by the explicit product. So the reduction is not merely "generate some local proof obligations": it generates a small fixed vocabulary of proof obligations, each with a specific semantic role to initialize, propagate, or exclude one dangerous local case.

Figure 3 complements Figure 2 by making the non-dangerous part of the reduction explicit. The upper half shows the two initialization triples generated once from the initial product configuration. The lower half shows the generic propagation pattern: for every well-formed product step, one triple propagates the user invariant and one propagates product-state coherence. The dangerous-step triple of Figure 2 is therefore not a standalone object; it is the local safety layer placed on top of this initialization-and-propagation discipline.

6 META-THEORY

The metatheory is organized around one main theorem and two adequacy theorems. The main theorem is soundness: validity of the generated local triples, together with semantic truth of the user invariants on admissible executions, implies conditional safety. The adequacy theorems explain

why the reduction is non-degenerate: under the appropriate semantic assumptions, the generated triples are themselves admissibly valid. The proof proceeds through a short chain of intermediate facts that correspond to the essential conceptual steps of the reduction.

The proof boundary is also explicit. The semantic core assumes only the truth of the generated triples and of user invariants on admissible executions when proving soundness. For relative completeness, the mechanized result is deliberately weaker: it establishes admissible-run validity, because global correctness constrains only inputs satisfying Avoid_A. No statement in this section depends on a concrete backend encoding, on a solver, or on a verification-condition language.

The structure is intentionally simple. First, one shows that every concrete tick matches the product step selected at that tick. Second, one shows that a realized step is structurally well formed. Third, one shows that the coherence facts carried by the local proof layer hold on executions. Fourth, one shows that a dangerous realized step activates the precondition of the corresponding safety triple. From there, soundness and relative completeness follow with little additional machinery.

Reading guide. The main theorem can be read backwards. To prove conditional safety, one assumes for contradiction that a guarantee violation occurs. Proposition 15 turns this global violation into one dangerous local step. The matching and coherence lemmas then show that the local precondition of the corresponding safety triple is indeed satisfied. Since the triple is valid, its postcondition $\bot$ cannot hold of any concrete successor, yielding the contradiction. The adequacy theorems later explain why the generated triples are not arbitrary local artifacts.

It is useful to read this section with two questions in mind: why local triple validity suffices for global conditional safety, and why the generated local triples are semantically legitimate once the global property is already known to hold.

For readability, we abbreviate the two global semantic assumptions used in the rest of the section as follows:

$$\mathsf{GlobCorr} \triangleq \forall u.\ \mathsf{Avoid}_A(u) \to \mathsf{Avoid}_G(u),$$
$$\mathsf{InvTrue} \triangleq \forall m_0 \in M, \forall u, \forall k \in \mathbb{N}.\ \mathsf{Avoid}_A(u) \to \mathsf{Inv}_{s_u(k)}(\mathsf{Ctx}_{m_0,u}(k)).$$

The first predicate is an entirely semantic statement about the program and the conditional specification. The second isolates the only genuinely external ingredient of the soundness and completeness arguments: user invariants are supplied with the specification, so their truth on admissible executions cannot be recovered from generated-triple validity alone.

Lemma 26 (Selected steps match the current context)**.** *For every $m_0 \in M$, every input stream u, and every tick $k \in \mathbb{N}$,*

$$\mathsf{Match}(\mathsf{Ctx}_{m_0,u}(k), \mathsf{step}_{m_0,u}(k)).$$

Proof idea. By definition, the selected product step at tick k is

$$\mathsf{step}_{m_0,u}(k) = (\mathsf{pst}_{m_0,u}(k), t_u(k), m_u(k), u(k), o_u(k), \mathsf{pst}_{m_0,u}(k+1)).$$

The tick context $\mathsf{Ctx}_{m_0,u}(k)$ contains exactly the same current components: current program state, current memory, current input, current output, current selected transition, and the current and next product-state coordinates. Therefore every equality required by the definition of $\mathsf{Match}(\mathsf{Ctx}_{m_0,u}(k), \mathsf{step}_{m_0,u}(k))$ holds by direct unfolding of the definitions. □

Lemma 27 (Realized steps are well formed)**.** *For every $m_0 \in M$, every input stream u, every tick $k \in \mathbb{N}$, and every product step p, if*

$$\mathsf{Match}(\mathsf{Ctx}_{m_0,u}(k), p),$$

then

$$\mathsf{WFStep}(p).$$

Proof idea. Assume $\mathsf{Match}(\mathsf{Ctx}_{m_0,u}(k), p)$ for

$$p = (p_{\mathrm{src}}, t, m, i, o, p_{\mathrm{dst}}).$$

By definition of matching, we have

$$p_{\mathrm{src}} = \mathsf{pst}_{m_0,u}(k), \qquad t = t_u(k), \qquad m = m_u(k), \qquad i = u(k), \qquad o = o_u(k),$$

and

$$p_{\mathrm{dst}} = \mathsf{pst}_{m_0,u}(k+1).$$

But by definition of the product-state run,

$$\mathsf{pst}_{m_0,u}(k+1) = \mathsf{Target}(\mathsf{pst}_{m_0,u}(k), t_u(k), m_u(k), u(k), o_u(k)).$$

Replacing the current quantities by the corresponding components of p gives

$$p_{\mathrm{dst}} = \mathsf{Target}(p_{\mathrm{src}}, t, m, i, o),$$

which is exactly $\mathsf{WFStep}(p)$. □

Lemma 28 (Coherence facts hold on executions)**.** *For every $m_0 \in M$, every input stream u, and every tick $k \in \mathbb{N}$,*

$$\mathsf{Coh}^{\mathsf{cur}}_{\mathsf{pst}_{m_0,u}(k)}(\mathsf{Ctx}_{m_0,u}(k)) \qquad \textit{and} \qquad \mathsf{Coh}^{\mathsf{next}}_{\mathsf{pst}_{m_0,u}(k+1)}(\mathsf{Ctx}_{m_0,u}(k)).$$

Proof idea. The current product state is defined from the current program state and the two automaton runs. The current projections of the tick context contain exactly these values, and its successor projections contain the values at tick $k+1$. Therefore the two coherence predicates hold by unfolding the definitions of the run and of the product state. □

Lemma 29 (A dangerous realized step activates a generated safety triple)**.** *For every $m_0 \in M$, every input stream u, every tick $k \in \mathbb{N}$, and every product step p, if*

$$\mathsf{Bad}(p) \wedge \mathsf{Match}(\mathsf{Ctx}_{m_0,u}(k), p) \wedge \mathsf{Inv}_{\pi_S(p_{\mathrm{src}})}(\mathsf{Ctx}_{m_0,u}(k)) \wedge \mathsf{Coh}^{\mathsf{cur}}_{p_{\mathrm{src}}}(\mathsf{Ctx}_{m_0,u}(k)),$$

then the precondition of the generated safety triple associated with p holds at tick k.

Proof idea. The precondition of $h^{\mathsf{NoBad}}(p)$ is exactly the conjunction of matching, the source-state user invariant, and source-state coherence. These three facts are hypotheses of the lemma. Moreover, realized steps are well formed by Lemma 27. Since $\mathsf{Bad}(p)$ also holds, the triple $h^{\mathsf{NoBad}}(p)$ is indeed one of the generated triples, and its precondition is satisfied at the current tick. □

6.1 Soundness

The first result justifies the reduction. Automaton coherence is reconstructed from the product semantics itself, whereas user invariants belong to the specification and are therefore assumed semantically true on admissible executions through $\mathsf{InvTrue}$.

Theorem 30 (Soundness of the reduction)**.** *Assume*

$$\forall h.\ \mathsf{Generated}(h) \rightarrow \mathsf{Valid}(h) \qquad \textit{and} \qquad \mathsf{InvTrue}.$$

Then every execution whose input stream satisfies the assumption also satisfies the guarantee:

$$\forall u.\ \mathsf{Avoid}_A(u) \Longrightarrow \mathsf{Avoid}_G(u).$$

Proof idea. The proof proceeds by contradiction. A global guarantee violation first localizes to one dangerous product step. Matching and automaton coherence then show that the precondition of the corresponding safety triple holds on the very tick that realizes this step; the remaining source-state

invariant fact is provided by InvTrue. Since that safety triple is valid and its postcondition is $\bot$, the dangerous tick cannot exist after all.

Proof. Fix an input stream u such that $\mathsf{Avoid}_A(u)$ holds. We must show that every execution on u satisfies G. Let therefore $m_0 \in M$ be arbitrary, and assume for contradiction that the execution from m_0 on u violates G.

By Proposition 15, there exists a tick $k \in \mathbb{N}$ such that the selected product step

$$p \triangleq \mathsf{step}_{m_0,u}(k)$$

is dangerous.

We first connect this concrete dangerous step to the local safety triple that was generated from it.

By Lemma 26, the current context matches this step:

$$\mathsf{Match}(\mathsf{Ctx}_{m_0,u}(k), p).$$

By Lemma 28, the same context satisfies the current coherence fact

$$\mathsf{Coh}^{\mathsf{cur}}_{p_{\mathsf{src}}}(\mathsf{Ctx}_{m_0,u}(k)).$$

By InvTrue and the standing assumption $\mathsf{Avoid}_A(u)$, the corresponding user-invariant clause also holds at tick k:

$$\mathsf{Inv}_{\pi_S(p_{\mathsf{src}})}(\mathsf{Ctx}_{m_0,u}(k)).$$

At this stage all ingredients of the safety-triple precondition are present on the concrete dangerous tick: matching, current coherence, and the user invariant at the source control state. Hence all conjuncts in the precondition of the generated safety triple $h^{\mathsf{NoBad}}(p)$ hold at the current context. By Lemma 29, this safety triple is one of the generated triples. By the first hypothesis

$$\forall h.\ \mathsf{Generated}(h) \to \mathsf{Valid}(h),$$

the triple $h^{\mathsf{NoBad}}(p)$ is valid. Since its target is the concrete program transition carried by p, triple validity applies to the concrete successor context $\mathsf{Ctx}_{m_0,u}(k+1)$. But the postcondition of $h^{\mathsf{NoBad}}(p)$ is $\bot$, which is impossible. This contradiction shows that no execution on u can violate G. Therefore $\mathsf{Avoid}_G(u)$ holds. □

The soundness proof factors through three conceptual ingredients: global guarantee violations localize to dangerous realized steps; automaton coherence and true user invariants guarantee that the safety-triple precondition holds on the dangerous tick; and a valid NoBad triple excludes any concrete realization of the corresponding dangerous step. The proof is therefore local once these three ingredients are in place.

6.2 Relative completeness

Intuition. Relative completeness runs in the opposite direction from soundness. Soundness starts from valid generated triples and concludes conditional safety. Relative completeness starts from semantic correctness of the program and asks whether the reduction generates semantically legitimate local proof obligations. For safety triples, this legitimacy follows directly from global correctness; for triples that transport user invariants, one must additionally assume that the user invariants are indeed true on admissible executions. The resulting notion is weaker than classical relative completeness of Hoare logic [10]: we do not claim that every semantically valid local triple is generated, nor that every valid generated triple is derivable in every backend. We only claim that the triples produced by the reduction are semantically valid on admissible runs under the natural hypotheses of the model.

Theorem 31 (Relative completeness of safety triples)**.** *If the program is globally correct, then every generated safety triple* NoBad *is admissibly valid.*

$$\mathsf{GlobCorr} \Longrightarrow \forall h.\ \mathsf{Generated}_{\mathsf{NoBad}}(h) \to \mathsf{ValidAdm}(h).$$

Proof idea. If one generated safety triple were not admissibly valid, then some admissible concrete execution would realize the dangerous step from which this triple was extracted. But a dangerous realized step is exactly a local witness of global guarantee violation under an admissible input. Global correctness therefore rules out the failure of any generated safety triple on admissible runs.

Proof. Assume GlobCorr. Let h be such that $\mathsf{Generated}_{\mathsf{NoBad}}(h)$ holds. By definition of generated safety triples, there exists a product step

$$p = (p_{\mathrm{src}}, t, m, i, o, p_{\mathrm{dst}})$$

such that $\mathsf{WFStep}(p)$, $\mathsf{Bad}(p)$, and $h = h^{\mathsf{NoBad}}(p)$.

We show that h is admissibly valid. Suppose on the contrary that it is not. Then, by definition of ValidAdm, there exist $m_0 \in M$, an input stream u, and a tick k such that

$$\mathsf{Avoid}_A(u), \qquad t_u(k) = t, \qquad \mathsf{Match}(\mathsf{Ctx}_{m_0,u}(k), p) \wedge \mathsf{Inv}_{\pi_S(p_{\mathrm{src}})}(\mathsf{Ctx}_{m_0,u}(k)) \wedge \mathsf{Coh}^{\mathsf{cur}}_{p_{\mathrm{src}}}(\mathsf{Ctx}_{m_0,u}(k))$$

hold.

Matching and coherence imply that the source and target of p coincide with the actual current and next product states of the execution at tick k. Since p is dangerous, its target guarantee component is bad while its target assumption component is not bad. Hence the execution on u violates G while preserving A, contradicting GlobCorr together with $\mathsf{Avoid}_A(u)$. Therefore h is admissibly valid. □

This theorem is the key non-degeneracy result for the safety part of the reduction. It explains why the soundness theorem is not vacuous: if the program is already globally correct, the reduction does not manufacture false NoBad triples that would make soundness trivially unusable.

Theorem 32 (Relative completeness of generated triples)**.** *Suppose that the program is globally correct and that the user invariants are semantically true on admissible executions. Then every generated triple is admissibly valid.*

$$\mathsf{GlobCorr} \wedge \mathsf{InvTrue} \Longrightarrow \forall h.\ \mathsf{Generated}(h) \to \mathsf{ValidAdm}(h).$$

Proof idea. This is a synthesis theorem over the five generated families. The two initialization triples are validated directly at the first tick. The two propagation families are validated by the coherence lemmas together with the truth of user invariants on admissible executions. The safety family is handled by Theorem 31. The complete result therefore follows by a simple case analysis on the generation relation.

Proof. Assume GlobCorr ∧ InvTrue, and let h be such that $\mathsf{Generated}(h)$ holds. By definition of Generated, there are five cases.

If $h = h^{\mathsf{init}}_{\mathsf{Inv}}$, admissible validity follows immediately from InvTrue at tick 0.

If $h = h^{\mathsf{init}}_{\mathsf{Coh}}$, admissible validity is immediate from the definition of the initial product state and of the initial tick context.

If $h = h^{\mathsf{prop}}_{\mathsf{Inv}}(p)$ for some well-formed step p, then its admissible validity follows from InvTrue: whenever the precondition holds on an admissible current context, the postcondition is exactly the invariant at the successor program state, which is true on the next tick of the execution.

If $h = h^{\mathsf{prop}}_{\mathsf{Coh}}(p)$ for some well-formed step p, then its admissible validity follows from Lemmas 26, 27, and 28: matching and current coherence identify the current product state with p_{src}, and

structural well-formedness of p forces its target to coincide with the next product state, so the postcondition $\mathrm{Coh}^{\mathrm{cur}}_{p_{\mathrm{dst}}}$ holds on the successor context.

Finally, if $h = h^{\mathrm{NoBad}}(p)$ for some dangerous well-formed step p, then admissible validity follows from Theorem 31. Thus all cases covered by $\mathrm{Generated}(h)$ are admissibly valid, as claimed. □

Remark 33. The dependence on user invariants is the only genuinely external dependency in the relative-completeness statement. This is expected: user annotations are not determined by the program and its temporal specification alone.

The automaton-coherence part is different. Because coherence is defined semantically from the actual current product state, the corresponding triples are relatively complete without any extra hypothesis beyond the semantics of the program model itself. This asymmetry between user invariants and automaton coherence is intentional: it isolates precisely which part of the theory depends on external annotations.

Taken together, the soundness and relative-completeness theorems justify the reduction itself. Soundness states that valid generated triples, together with true user invariants on admissible executions, are sufficient for global conditional safety. Relative completeness states that, under the appropriate semantic hypotheses, the reduction does not generate arbitrary false local proof objects on admissible runs. This is the mathematically relevant notion of adequacy for the reduction layer, independently of any later backend proof search.

7 INSTANTIATION AND BACKEND REFINEMENTS

The theory above is independent of any particular specification language or backend. A concrete tool may instantiate it as follows.

This section is deliberately secondary to the core theory. Its role is to show that the abstract product, clauses, and triples are concrete enough to be realized in an existing synchronous toolchain. It does not introduce new proof objects or new proof principles.

The point is therefore not to re-prove the theory on a concrete language. The point is to show that the abstract local proof objects described above are recognizable in a realistic synchronous toolchain, rather than being purely existential artifacts of the semantics.

First, a temporal specification language can be restricted to a safety fragment and compiled to safety automata. This step is standard [17, 21]. Second, the generated triples can be encoded in a local proof language or a verification-condition generator. In one possible instantiation, an external deductive backend such as Why3 is used to discharge the resulting local proof obligations [13]. Backend optimizations, such as grouping multiple triples that share the same program transition, belong to this refinement layer and do not change the mathematical core presented in this paper.

The refinement boundary is therefore explicit. The abstract theory stops at three objects: semantic clauses, relational triples, and validity of those triples. A concrete tool must additionally choose a specification syntax, a formula language, a verification-condition encoding, and a proof backend. These choices matter for automation, but not for the soundness and relative completeness theorems stated above.

The existing Kairos tool is best understood as one such instantiation. It chooses a concrete safety-LTL fragment, a concrete encoding discipline for traces and invariants, and a Why3-based validation backend. These choices are important for practice, but they are secondary with respect to the abstract reduction proved here.

It is nevertheless useful to spell out this refinement boundary more concretely. In the abstract theory, the temporal specification is already given by two safety automata and the proof layer is already given by relational triples. A concrete tool must additionally answer three implementation

questions. It must first choose a source specification language and compile it to safety automata. It must then decide how the abstract triples are rendered as local transition contracts. Finally, it must choose a backend validator and an encoding of these contracts for that backend. The purpose of the present section is not to reprove the theory in a tool-specific setting, but to show that one concrete tool chain does instantiate exactly these three choices.

7.1 A concrete Kairos instance of the running example

To make the instantiation explicit, we now return to the same running example of Section 3, this time in the concrete syntax of the current Kairos prototype. The following node is not a second example; it is the tool-level realization of the same resettable delay used throughout the paper:

```
node resettable_delay(reset: int, x: int) returns (y: int)
contracts
  requires: G((reset = 0 or reset = 1) and (reset = 1 => x = 0));
  ensures: G((reset = 1) => (y = 0));
  ensures: X G((reset = 0 and prev reset = 1) => y = 0);
  ensures: X G((reset = 0 and prev reset = 0) => y = prev x);
locals
  m : int;
states
  Init(init), Run;
invariants
  in Run:
    (prev reset = 1 => m = 0) and
    (prev reset = 0 => m = prev x);
transitions
  Init:
    to Run when reset = 1 { y := 0; m := 0; }
    to Run when reset = 0 { y := 0; m := x; }
  Run:
    to Run when reset = 1 { y := 0; m := 0; }
    to Run when reset = 0 { y := m; m := x; }
end
```

The example can be read almost operationally. The contracts block contains the temporal assumption and guarantee clauses. The locals and states blocks declare the program memory and control states. The invariants block attaches user invariants to a control state, here Run. Finally, each clause in the transitions block describes one synchronous tick: when its guard is satisfied in the current state, the block computes the current output and the next memory. Thus the mathematical step relation of the earlier sections corresponds here to one guarded transition clause together with its assignments.

From the viewpoint of synchronous compilation, this concrete organization is in the spirit of OBC-style intermediate languages derived from Lustre-like compilers: control states are explicit, local memory is explicit, and each tick executes one guarded transition whose effect is entirely local to the current instant [7, 19, 24]. This is exactly the kind of representation for which the abstract reduction of the paper is meant to provide a backend-independent proof interface.

This concrete syntax is richer than the abstract presentation used earlier, but the mathematical roles are the same. The environment assumption forces ‘reset‘ to behave as a Boolean flag and, when ‘reset = 1‘, constrains the input to carry the reinitialization value ‘x = 0‘. The three guarantee clauses split the intended safety property into the reset case, the first non-reset step after a reset,

and the ordinary delayed case. The user invariant records the meaning of the memory cell across these two operational modes. From this specification, the tool builds one safety automaton for the assumption and one for the guarantee, forms the explicit product with the program, and extracts the local triples justified by the abstract theory.

Concretely, the source language accepted by the current tool uses a safety fragment of LTL with past operators such as prev. The compilation pipeline first propositionalizes the atomic predicates that occur in the temporal formulas and then constructs deterministic residual monitors for the assumption and the guarantee. This is a standard monitor-oriented compilation strategy for safety fragments [17, 21]. For the present running example, the current tool computes two residual states for the assumption automaton and three residual states for the guarantee automaton. The reachable explicit product with the program contains five states.

The point of this compilation is not just to recover a finite safety automaton, but to obtain exactly the abstract ingredients used in the metatheory. Each reachable product step carries an assumption edge, a guarantee edge, and a program transition. The generated triples are then organized according to the same five families as in Section 5: one initial coherence triple for the automaton side, one initial triple for the user invariant, propagation triples for automaton coherence, propagation triples for user invariants, and one safety triple for each dangerous product step. On the current running example, the concrete generation produces 22 triples in total: 3 safety triples and 19 coherence triples, of which 1 is initial and 18 are propagation triples.

At the level of generated proof objects, nothing fundamentally new happens here: the concrete pipeline merely instantiates the five abstract families already defined in Section 5. The initial triples become initial verification conditions. The propagation triples become local transition contracts whose preconditions mention both the current product state and the current invariant fact. The dangerous-step triples become transition contracts with postcondition $\bot$. What changes from the abstract layer to the concrete one is therefore not the proof interface itself, but the surface syntax in which the same five families are expressed.

The decorated intermediate program makes this correspondence visible. The following excerpt is a readability-preserving normalization of the generated OBC-like code for the Run –> Run non-reset transition. In the actual generated artifact, the formulas written here as Φ_A and Φ_{bad} are fully expanded Boolean combinations of local assumption-side and bad-guarantee cases. We use abbreviations here only to keep the local structure legible on the page.

```
transition Run -> Run when reset = 0 {
  require __aut_state = Aut1 or __aut_state = Aut2 {Compatibility};
  require __aut_state = Aut2 -> Phi_A              {AssumeAutomaton};
  require (pre(reset) = 1 -> m = 0)
       and (pre(reset) = 0 -> m = pre(x))          {Coherency};
  ensure  __aut_state = Aut2 -> not Phi_bad        {Instrumentation};
  ensure  (reset = 1 -> m = 0)
       and (reset = 0 -> m = x)                    {Coherency};
  do
    y := m;
    m := x;
}
```

Here Φ_A abbreviates the disjunction of local assumption-automaton cases compatible with the current transition, and Φ_{bad} abbreviates the local bad-guarantee case for this transition. The fully expanded version appears in the generated artifact; the present display keeps only the contract shape. The four annotations have a direct semantic reading. The `Compatibility` requirement is the concrete image of the current-state coherence triple: it says that the automaton state stored in the decorated program agrees with the product source state. The `AssumeAutomaton` requirement is the concrete image of the automaton coherence triple that propagates assumption-side information

extracted from the product. The `Coherency` postcondition is the image of the user invariant propagation triple. Finally, the `Instrumentation` postcondition is the image of the NoBad triple: for this transition, the concrete contract excludes the local combination of assumption and guarantee edges that would lead the guarantee automaton to its bad state.

This is the most informative transition of the running example because it makes the temporal nature of the proof obligations explicit. The precondition no longer depends only on the current program memory: it mentions `pre` operators, so that the local contract can talk about the previous reset mode and the previous input value. This is exactly the information needed to justify the ordinary-delay branch of the guarantee and to rule out the dangerous case in which the current output would disagree with the previous non-reset input. In the current implementation, the corresponding fully expanded artifact can be read in the generated `.obc+` file for the running example.

This excerpt also explains why the abstract theory is phrased in terms of triples rather than isolated formulas. In the generated code, all four facts are attached to the same transition contract. The deductive backend therefore reasons about them in a single local proof context, exactly as intended by the abstract reduction.

For the running example, one can read the concrete generation scheme directly:

$$\begin{aligned} \text{reset tick} &\rightsquigarrow \text{coherence propagation + invariant propagation,} \\ \text{non-reset tick from Init} &\rightsquigarrow \text{the same two propagation triples,} \\ \text{non-reset tick from Run} &\rightsquigarrow \text{the same propagation triples, plus possibly } h^{\mathrm{NoBad}}(p). \end{aligned}$$

This is the concrete reflection of the abstract chain

$$\text{global violation} \Longrightarrow \text{dangerous step} \Longrightarrow \text{NoBad clause} \Longrightarrow h^{\mathrm{NoBad}}(p).$$

The backend does not rediscover this structure; it only receives the local triples that the reduction has already determined.

This is precisely the point where the running example becomes useful again. A non-reset tick with an incorrect output still corresponds to a dangerous product step, but the concrete source makes visible why two different families of coherence triples are needed. The reset discipline appears in the automaton side of the product, while the relation between ‘m‘ and the recent history of ‘x‘ appears in the user-invariant side. In the concrete tool chain, the dangerous step yields a local safety triple whose precondition combines both forms of coherence. The tool then translates these triples to the input language of a backend validator.

The current implementation renders these triples as transition-local contracts for a verification-condition generator. In the present instantiation, the backend is Why3 [13]. The concrete pipeline emits a decorated intermediate program, a Why3 file, and the resulting verification conditions. Temporal operators do not survive there as primitive connectives. Instead, constructs such as $\mathrm{pre}(x)$ or $\mathrm{pre}(reset)$ are compiled into ordinary first-order state by introducing explicit history variables together with synchronous update equations. The verification conditions that Why3 sees are therefore standard first-order preconditions and postconditions over the current variables, the control state, the automaton state, and these finite history variables. The temporal reading lives at the level of the abstract reduction; the backend only receives its first-order image.

This is also how the abstract shift operation is instantiated concretely. In the running example, the coherence postcondition for the non-reset `Run` $\longrightarrow$ `Run` transition says that after the tick the remembered value must agree with the current input. Abstractly this is written as a shifted obligation about the successor context. Concretely, the backend realizes this shift by talking about the history variables that represent the next one-step past: the current postcondition updates the state so that the next precondition can read `__pre_x` and `__pre_reset` in ordinary first-order logic.

In this way, the abstract temporal shift is compiled into synchronous state updates plus first-order references to finite history.

The essential point is that this stage does not introduce new semantic proof objects: it only re-expresses the same five generated families in a backend language. In particular, the current backend still contains an explicit initial coherence goal for the automaton part, transition contracts corresponding to propagation triples, and transition contracts with postcondition $\bot$ corresponding to safety triples.

For the running example, the generated Why3 file is successfully discharged by Z3 through Why3 transformations. Using Why3 with the `simplify_formula`, `eliminate_if_term`, `remove_unused`, and `split_vc` transformations, a `30s` timeout per goal, Why3 1.8.2, Z3 4.15.2, and the local development machine (Apple M2, 8 cores, 16 GB RAM, macOS 15.6 / Darwin 24.6.0), the full generated file is discharged in about `4.4s` of wall-clock time. This timing is only a reproducibility indicator for the running example, not a performance claim about the method. It does not strengthen the abstract soundness theorem, but it shows that the proof interface isolated in the paper is concrete enough to be realized by an existing deductive backend. It also illustrates the intended separation of concerns: the paper proves properties of the reduction itself, whereas the backend merely validates one concrete encoding of the resulting local triples.

This reading also clarifies what the paper does *not* claim. It does not claim completeness of a concrete solver, completeness of Why3 encodings, or a backend-independent procedure for discharging every valid triple automatically. The abstract theory justifies the reduction; concrete automation remains a refinement problem.

7.2 A more data-intensive Kairos example

The running example remains intentionally small. This is useful for exposition, but it does not fully illustrate one practical strength of the reduction: the program state itself need not be finite. To make this point concrete, the current Kairos prototype also verifies the following `credit_balance_monitor` node, whose control structure is tiny but whose semantic state ranges over unbounded integers:

```
node credit_balance_monitor(reset: int, credit: int, debit: int)
returns (bal: int)
contracts
  requires: G((reset = 0 or reset = 1)
           and (credit >= 0) and (debit >= 0));
  requires: G((reset = 1) => (credit = 0 and debit = 0));
  ensures: G((reset = 1) => (bal = 0));
  ensures: G(bal >= 0);
locals
  m : int;
states
  Init(init), Run;
invariants
  in Run:
    (m >= 0) and (m = prev bal);
transitions
  Init:
    to Run when reset = 1 {
      bal := 0;
      m := 0;
    }
    to Run when reset = 0 and debit <= credit {
```

```
      bal := credit - debit;
      m := credit - debit;
    }
  Run:
    to Run when reset = 1 {
      bal := 0;
      m := 0;
    }
    to Run when reset = 0 and debit <= m + credit {
      bal := m + credit - debit;
      m := m + credit - debit;
    }
end
```

This node is still a safety problem, but it is no longer naturally viewed as a finite-state verification instance. The balance evolves over the integer domain without any a priori bound, so explicit-state model checking would require a separate abstraction argument before it could even recover a finite state space. By contrast, the reduction of the present paper keeps the automaton side finite while leaving the program memory symbolic. The user invariant $(m >= 0)$ and $(m = \text{prev bal})$ provides the local arithmetic summary needed to show that every reachable non-reset transition preserves $\text{bal} >= 0$ under the transition guard.

Conceptually, this second example does not change the proof architecture at all. The program still induces an explicit product with assumption and guarantee automata, dangerous steps still localize global guarantee violations, and the resulting obligations are still discharged as local transition proofs. What changes is only the character of the program state: the example stresses that the method is not tied to finite-state program semantics even though the specification side continues to use finite safety automata.

8 RELATED WORK

The paper sits at the intersection of four well-established lines of work.

The first line is the classical theory of temporal properties on infinite traces. The distinction between safety and liveness goes back to Alpern and Schneider [1], while temporal logic as a specification language goes back to Pnueli [23]. Our setting is more specific: it is a conditional safety setting, where the property of interest is not unconditional trace acceptance but a guarantee that must hold only under an explicit environment assumption.

The second line is synchronous and reactive verification. The view of programs as deterministic stream transformers is classical in synchronous programming [4, 19], and observer-based reasoning is a standard verification technique in that community [7]. Deductive verification of synchronous programs also has an established lineage in the Lustre community, notably through the Lustre/PVS line of work developed by Bensalem, Caspi, Dumas, and coauthors [3, 11, 12]. For synchronous programs with rich data domains, Gesell and Schneider observe that “model checking suffers from the so-called state-space explosion problem” and that “large and data-intensive synchronous programs requires further techniques” [18].

What is less standard here is the semantic role given to the explicit product. In the present paper the product is not merely an executable observer or a model-checking artifact. It is the canonical semantic object from which dangerous local situations, clauses, and local proof objects are defined.

The third line is deductive reasoning and relative completeness. Hoare logic and Cook’s relative completeness theorem establish the classical template “semantic validity implies derivability” for local proof systems [10, 20]. The present paper uses relational triples as the local proof layer, but its

relative-completeness results live one level above the proof system itself: they justify the reduction that generates these local triples from global conditional-safety semantics. In that sense, the work is complementary to the classical completeness story for Hoare logic rather than a replacement for it. In a similar spirit, Fix and Grumberg describe temporal deduction as a reduction that "reduces temporal verification to assertional reasoning" [14].

The fourth line is the compilation of temporal specifications toward program verification or runtime reasoning. On the deductive side, Aorai and CaFE show that temporal properties can be compiled into local program-level proof artifacts for C verification backends [15, 16]. On the synchronous and contract-based side, AGREE and Kind 2 show how rich reactive contracts can be checked compositionally and automatically [2, 8]. On the monitoring side, systems such as Copilot compile temporal or stream specifications into runtime monitors [22]. These works make clear that temporal reasoning and local proof obligations can be combined successfully in practice. The earlier Lustre/PVS work cited above is particularly relevant here because it already treats synchronous programs, deductive proof, and generated proof obligations in a single setting, even though the semantic reduction developed in the present paper is organized around an explicit assumption/guarantee product rather than directly around a proof backend. A complementary precursor is the STeP line of work, which already combines "deductive and algorithmic verification" and explicitly targets reactive systems with "infinite data domains" [5]. At a more semantic level, Yahav *et al.* emphasize the limits of "classical model checking, which uses propositional temporal logic" once temporal properties range over richer unbounded state [25].

The present contribution differs from these systems in two ways. First, the paper is centered on a semantic reduction theorem rather than on a concrete verification backend or observer-construction pipeline. Second, the explicit product is used as the semantic unit of localization: dangerous steps, semantic clauses, and relational triples are defined from the product itself rather than being introduced only as verification-condition engineering devices. This is why the paper is best read not as a new backend architecture, but as a metatheoretic account of how conditional safety on reactive programs can be reduced to local proofs.

9 MECHANIZATION

The metatheory of the paper is formalized in an independent Rocq development located in `specv2/rocq/`. The development is intentionally small and follows the conceptual structure of the paper rather than the architecture of a particular tool chain. In its current form it contains eight files and 1217 non-comment, non-empty lines. Counting lines outside proof blocks as specification and lines from `Proof.` to `Qed./Defined.` as proof, the development contains 730 specification lines and 487 proof lines.

The structure of the development mirrors the mathematical decomposition used in the paper: `ReactiveModel.v` defines reactive ticks, `ConditionalSafety.v` defines the specification layer, `ExplicitProduct.v` defines product states and product steps, `GeneratedClauses.v` defines semantic clauses, `RelationalTriples.v` defines the local proof objects, and `Soundness.v` and `RelativeCompleteness.v` formalize the main results. The file `ResettableDelayExample.v` instantiates the abstract theory on the running example and concludes `resettable_delay_correct` by proving the generated triples valid and then applying the abstract soundness theorem.

The purpose of the mechanization is to support the mathematical claims of the paper, not to introduce a second technical narrative. For this reason, the paper keeps the proof ideas in the main text and only records the paper-to-Rocq correspondence here:

Rocq file	Spec	Proof
`ReactiveModel.v`	73	12
`ConditionalSafety.v`	53	23
`ExplicitProduct.v`	153	40
`GeneratedClauses.v`	69	0
`RelationalTriples.v`	115	13
`Soundness.v`	34	46
`RelativeCompleteness.v`	51	89
`ResettableDelayExample.v`	182	264
Total	**730**	**487**

Overall, this yields 1217 non-comment, non-empty lines split into 730 specification lines and 487 proof lines.

This separation is intentionally close to the human proof structure. The Rocq development contains additional internal lemmas and explicitly parameterized semantics for arbitrary initial memories, but the public theorem layer follows the same organization as the paper. The concrete example file is the only place where the abstract theory is instantiated on one fixed program; the remaining files stay fully parametric.

10 CONCLUSION

We have presented a semantic reduction from conditional safety of reactive programs to local relational proofs. The reduction is centered on an explicit product between the program and two safety automata, one for assumptions and one for guarantees. This product localizes global violations through dangerous steps. From these steps we extract semantic clauses and package them into relational Hoare triples over ticks.

The resulting theory yields three central results: soundness, relative completeness of the safety reduction on admissible runs, and relative completeness of all generated triples on admissible runs under true user invariants. The theory is independent of any particular tool. Concrete compilers, encodings, and solver backends refine this core but do not define it.

The conceptual point is that explicit products, dangerous steps, semantic clauses, and relational triples form a small but sufficient vocabulary for this reduction. Once these objects are fixed, the main proofs become short and their hypotheses become transparent. This is the sense in which the paper aims at a general reduction theorem rather than a tool-specific account.

Future work. The next step is not to complicate the core theory, but to refine its instantiations. A first direction is to widen the specification side while preserving the same local-proof interface, for instance by clarifying which larger safety fragments still compile to automata and generated triples in a principled way. A second direction is to tighten the bridge to proof-producing backends, so that the semantic clauses and generated triples of the paper map more directly to concrete verification conditions and proof certificates. A third direction is to systematize language-level embeddings, in particular for synchronous languages with richer state structure than the running example, while keeping the abstract reduction independent of any one source language.

REFERENCES

[1] Bowen Alpern and Fred B. Schneider. 1985. Defining Liveness. *Inform. Process. Lett.* 21, 4 (1985), 181–185. https://doi.org/10.1016/0020-0190(85)90056-0

[2] John Backes, Darren Cofer, Andrew Gacek, and Michael Whalen. 2021. *The Assume Guarantee Reasoning Environment.* Technical Report. Loonwerks. https://loonwerks.com/publications/pdf/backes2021techreport.pdf

[3] Saddek Bensalem, Paul Caspi, Cécile Dumas, and Catherine Parent-Vigouroux. 1999. A Methodology for Proving Control Systems with Lustre and PVS. In *Dependable Computing for Critical Applications—7 (Dependable Computing*

and Fault Tolerant Systems, Vol. 12), Charles B. Weinstock and John Rushby (Eds.). IEEE Computer Society, San Jose, CA, USA, 89–107. https://www-verimag.imag.fr/details.html?lang=&pub_id=BenCasDumPar99

[4] Albert Benveniste and Gérard Berry. 1991. The Synchronous Approach to Reactive and Real-Time Systems. *Proc. IEEE* 79, 9 (1991), 1270–1282. https://doi.org/10.1109/5.97297

[5] Rudolf Berghammer, Yassine Lakhnech, Zohar Manna, Nikolaj S. Bjørner, Anca Browne, Michael Colón, Bernd Finkbeiner, Mark Pichora, Henny B. Sipma, and Tomás E. Uribe. 1999. An Update on STeP: Deductive-Algorithmic Verification of Reactive Systems. In *Tool Support for System Specification, Development and Verification*. Springer Vienna, Vienna, 174–188. https://doi.org/10.1007/978-3-7091-6355-9_13

[6] Ahmed Bouajjani, Javier Esparza, and Oded Maler. 1997. Reachability Analysis of Pushdown Automata: Application to Model-Checking. In *Concurrency Theory (Lecture Notes in Computer Science, Vol. 1243)*. Springer, Berlin, Heidelberg, 135–150. https://doi.org/10.1007/3-540-63141-0_10

[7] Paul Caspi, Adrien Curic, Antoine Maignan, and Stavros Tripakis. 1999. Formal Design of Distributed Control Systems with Lustre. In *Computer Safety, Reliability and Security (Lecture Notes in Computer Science, Vol. 1698)*. Springer, Berlin, Heidelberg, 396–409. https://doi.org/10.1007/3-540-48249-0_34

[8] Adrien Champion, Alain Mebsout, Christoph Sticksel, and Cesare Tinelli. 2016. The Kind 2 Model Checker. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 9780)*. Springer, Cham, 510–517. https://doi.org/10.1007/978-3-319-41540-6_29

[9] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 1855)*. Springer, Berlin, Heidelberg, 154–169. https://doi.org/10.1007/10722167_15

[10] Stephen A. Cook. 1978. Soundness and Completeness of an Axiom System for Program Verification. *SIAM J. Comput.* 7, 1 (1978), 70–90. https://doi.org/10.1137/0207005

[11] Cécile Dumas and Paul Caspi. 2000. A PVS Proof Obligation Generator for Lustre Programs. In *Logic for Programming and Automated Reasoning (Lecture Notes in Artificial Intelligence, Vol. 1955)*. Springer, Berlin, Heidelberg, 179–188. https://www-verimag.imag.fr/details.html?lang=&pub_id=lpar

[12] Cécile Dumas Canovas. 2000. *Méthodes déductives pour la preuve de programmes Lustre*. Ph. D. Dissertation. Université Joseph Fourier. https://www-verimag.imag.fr/details.html?lang=&pub_id=cecileDumas

[13] Jean-Christophe Filliâtre and Andrei Paskevich. 2013. Why3 — Where Programs Meet Provers. In *Programming Languages and Systems (Lecture Notes in Computer Science, Vol. 7792)*. Springer, Berlin, Heidelberg, 125–128. https://doi.org/10.1007/978-3-642-37036-6_8

[14] Limor Fix and Orna Grumberg. 1996. Verification of Temporal Properties. *Journal of Logic and Computation* 6, 3 (1996), 343–361. https://doi.org/10.1093/logcom/6.3.343

[15] Frama-C Team. 2022. Aoraï Plugin Tutorial. https://frama-c.com/download/aorai-manual-23.0-rc1-Vanadium.pdf. Frama-C documentation.

[16] Frama-C Team. 2026. CaFE: CaRet Frama-C Extension. https://www.frama-c.com/fc-plugins/cafe.html. Frama-C plugin documentation.

[17] Paul Gastin and Denis Oddoux. 2001. Fast LTL to Büchi Automata Translation. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 2102)*. Springer, Berlin, Heidelberg, 53–65. https://doi.org/10.1007/3-540-44585-4_6

[18] Manuel Gesell and Klaus Schneider. 2012. A Hoare Calculus for the Verification of Synchronous Languages. In *Proceedings of the 6th Workshop on Programming Languages Meets Program Verification (PLPV '12)*. Association for Computing Machinery, New York, NY, USA, 37–48. https://doi.org/10.1145/2103776.2103782

[19] Nicolas Halbwachs. 1993. *Synchronous Programming of Reactive Systems*. Springer, Dordrecht. https://doi.org/10.1007/978-1-4757-2231-4

[20] C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580. https://doi.org/10.1145/363235.363259

[21] Orna Kupferman and Moshe Y. Vardi. 2001. Model Checking of Safety Properties. *Formal Methods in System Design* 19, 3 (2001), 291–314. https://doi.org/10.1023/A:1011254632723

[22] Lee Pike, Nis Wegmann, Sebastian Niller, and Alwyn Goodloe. 2013. Copilot: Monitoring Embedded Systems. *Innovations in Systems and Software Engineering* 9, 4 (2013), 235–255. https://doi.org/10.1007/s11334-013-0223-x

[23] Amir Pnueli. 1977. The Temporal Logic of Programs. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS '77)*. IEEE Computer Society, Los Alamitos, CA, USA, 46–57. https://doi.org/10.1109/SFCS.1977.32

[24] Verimag Lustre Team. 2022. The Lustre V6 Reference Manual. https://www-verimag.imag.fr/DIST-TOOLS/SYNCHRONE/lustre-v6/doc/lv6-ref-man.pdf. Documentation manual.

[25] Eran Yahav, Thomas Reps, Mooly Sagiv, and Reinhard Wilhelm. 2006. Verifying Temporal Heap Properties Specified via Evolution Logic. *Logic Journal of the IGPL* 14, 5 (2006), 755–783. https://doi.org/10.1093/jigpal/jzl009